ENGINEERING BLUEPRINT · PLAYBOOK · COOKBOOK · RUNBOOK

The Agentic FDE Lifecycle

An AI-Native Software Development Lifecycle for Embedded & Remote Forward-Deployed Engineering PODs

---

Purpose: Define and operationalize an agentic SDLC that integrates autonomous AI agents into forward-deployed engineering. It shifts human roles from manual coding to system architecture, guardrail configuration, and review — and scales across a multi-cloud (generic, AWS, GCP, Azure) tooling ecosystem for the widest possible industry reach.

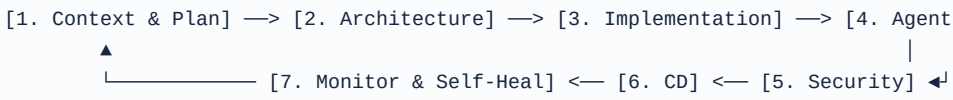
Two topologies, one codebase: Embedded PODs (human-in-the-loop agents inside existing teams) and Remote PODs (isolated, fully autonomous agent clusters) run as a parallel rollout — letting risk-averse teams adopt supervised agents immediately while modern teams deploy autonomous micro-services.

THE OPERATING THESIS

The model proposes; a deterministic gate disposes. Agents execute the lifecycle iteratively — pulling tasks from a backlog and pushing verified code to production — while humans own architecture, guardrails, and the approval surface. Embedded earns trust; Remote scales it. Cloud-agnostic by design, cloud-native by integration.

Autonomous agents execute this lifecycle iteratively, pulling tasks from a backlog and pushing verified code to production. The loop is closed: production telemetry feeds back into planning, so the system self-heals and re-plans without a human restart.

The closed-loop agentic SDLC



The seven stages

#	Stage	What the agents do
1	**Context Discovery & Planning**	Ingest Jira/Linear issues, analyze codebase topology, and generate an execution plan.
2	**Architecture & Design**	Tool-use agents query documentation, generate API specs, and verify schema compatibility.
3	**Implementation & Coding**	Multi-agent coding squads write code, resolve merge conflicts, and self-correct syntax errors.
4	**Agentic Verification (QA)**	Autonomous generation of unit, integration, and end-to-end tests from the acceptance criteria.
5	**Security & Guardrails**	Automated SAST and compliance checking via specialized agent tools.
6	**Continuous Deployment**	Automated PR management, canary deployments, and rolling updates.
7	**Observability & Self-Healing**	Monitoring agents detect anomalies, map stack traces, and auto-generate hotfix PRs.

To scale agentic engineering, deploy a unified stack that supports long-running execution, vector memories, and sandboxed code execution — cloud-agnostic at the core, with native bindings into each major cloud's agentic services.

Core architecture frameworks

- **Multi-agent orchestration:** LangGraph (cyclic, stateful workflows), CrewAI (role-based squads), or AutoGen (event-driven conversations).
- **Code-specific agent frameworks:** Aider (git-integrated coding), SWE-agent (sandboxed software-engineering environment), or OpenHands (open-source agent workspace).

Automated engineering infrastructure

Lifecycle stage	Tool category	Examples
Context & Retrieval	Codebase graph vector DB	Bloop, Greptile, Sweep, Qdrant
Execution Sandbox	Secure compute runtimes	E2B, Fly.io Sandboxes, Docker
Code Generation	Advanced LLM coding models	Claude 3.5 Sonnet, GPT-4o, DeepSeek-Coder
Testing & Evaluation	Agentic testing frameworks	CodiumAI / Qodo, agent-eval harnesses

The universal multi-cloud agentic ecosystem

Remain cloud-agnostic while natively integrating each provider's specialized agentic services. One orchestration codebase; four interchangeable execution substrates.

Pillar	Generic / OSS	AWS	GCP	Azure
Orchestration	LangGraph, CrewAI, AutoGen	Amazon Bedrock Flows	Vertex AI Agent Builder	Azure AI Studio Orchestrator
Compute Sandbox	E2B, Docker, Fly.io	Lambda / ECS Run	Cloud Run v2 (gVisor)	Container Apps Dynamic Sessions
Code / Context DB	Greptile, Qdrant	Bedrock Knowledge Bases	Vertex AI Vector Search	Azure AI Search (Vector)
Primary Code Models	DeepSeek-Coder, Llama-3	Anthropic Claude 3.5	Gemini 1.5 Pro / Ultra	Azure OpenAI GPT-4o / o1

Parallel rollout: the two topologies

Deploy both topologies simultaneously from a single, unified orchestration codebase. Risk-averse teams adopt human-in-the-loop agents; modern teams deploy fully autonomous micro-services.

One backlog, two execution topologies

A) Embedded FDE PODs — Human-in-the-Loop

Embedded PODs place AI agents directly inside human engineering teams. The agents act as force multipliers, taking instructions from human developers and halting at an approval gate before merge.

Cookbook A — Embedded PR-review guardrail agent (LangGraph)

```
from langgraph.graph import StateGraph, END
from typing import Dict, TypedDict

class PRState(TypedDict):
    pr_diff: str
    lint_passed: bool
    review_comments: str
    approved: bool

def run_linter_agent(state: PRState) -> Dict:
    # Simulating a sandboxed linting execution tool
    diff = state["pr_diff"]
    has_errors = "console.log" in diff   # Basic rule example
    return {"lint_passed": not has_errors}

def human_review_gate(state: PRState) -> Dict:
    # Halts execution to await a webhook from a human reviewer
    if not state["lint_passed"]:
        return {"approved": False,
                "review_comments": "Fix linter errors first."}
    return {"approved": True}   # else request human sign-off

workflow = StateGraph(PRState)
workflow.add_node("LinterAgent", run_linter_agent)
workflow.add_node("HumanGate", human_review_gate)
workflow.set_entry_point("LinterAgent")
workflow.add_edge("LinterAgent", "HumanGate")
workflow.add_edge("HumanGate", END)
embedded_pod = workflow.compile()
```

B) Remote FDE PODs — Fully Autonomous

Remote PODs operate as isolated, sovereign micro-services. They consume API requirements, build the service from scratch in a sandbox, run verification, and expose an endpoint — with zero human code review, falling back to a self-healing loop on failure.

```
Cookbook B — Remote micro-service agent blueprint (E2B sandbox)
from e2b_code_interpreter import Sandbox

def deploy_remote_service_pod(prompt_specification: str):
    """Spin up an isolated sandbox to build & test a service."""
    with Sandbox() as mx_sandbox:
        # 1. Agent writes code to the sandbox filesystem
        mx_sandbox.files.write("app.py", """
from fastapi import FastAPI
app = FastAPI()
@app.get("/health")
def health(): return {"status": "healthy"}
        """)
        # 2. Agent installs deps & runs verification autonomously
        mx_sandbox.commands.run("pip install fastapi uvicorn")
        execution = mx_sandbox.commands.run("pytest app.py")

        if execution.exit_code == 0:
            print("Verified. Deploying to Kubernetes...")
            # Trigger CD pipeline API call here
        else:
            print("Tests failed. Activating self-healing loop...")
            # Feed execution.stderr back into the LLM to rewrite app.py
```

C) Multi-Cloud Parallel Orchestration

A unified, cloud-agnostic graph routes each task to either an Embedded POD (human review) or a Remote POD (automated cross-cloud sandboxes), selecting the provider substrate at runtime.

```
Cookbook C — Cross-cloud unified router (Embedded + Remote)
from typing import Dict, TypedDict
from langgraph.graph import StateGraph, END

class PODState(TypedDict):
    task_type: str        # "embedded" or "remote"
    target_cloud: str     # "aws", "gcp", or "azure"
    codebase_diff: str
    execution_logs: str
    verified: bool

def run_cloud_sandbox(state: PODState) -> Dict:
    cloud = state["target_cloud"]
    print(f"[Sandbox] Init isolated compute on {cloud.upper()}...")
    if cloud == "aws":    pass   # AWS Lambda / ECS sandbox
    elif cloud == "gcp":  pass   # Google Cloud Run (gVisor)
    elif cloud == "azure":pass   # Azure Container Apps Sessions
    return {"execution_logs": "Build OK. Tests passed.",
            "verified": True}

def embedded_human_gate(state: PODState) -> Dict:
    print("[Embedded] Pausing for human code review...")
    return {"verified": True}   # webhook to GitHub PR / Slack

def remote_autonomous_gate(state: PODState) -> Dict:
    print("[Remote] Autonomous checks + AI load testing...")
    return {"verified": state["verified"]}

def route_pod_type(state: PODState) -> str:
    return "HumanApproval" if state["task_type"] == "embedded" \
           else "AutonomousVerification"

workflow = StateGraph(PODState)
workflow.add_node("CloudSandbox", run_cloud_sandbox)
workflow.add_node("HumanApproval", embedded_human_gate)
workflow.add_node("AutonomousVerification", remote_autonomous_gate)
workflow.set_entry_point("CloudSandbox")
workflow.add_conditional_edges("CloudSandbox", route_pod_type, {
    "HumanApproval": "HumanApproval",
    "AutonomousVerification": "AutonomousVerification" })
workflow.add_edge("HumanApproval", END)
workflow.add_edge("AutonomousVerification", END)
fde_orchestrator = workflow.compile()
```

A step-by-step operating guide to provision, monitor, and govern the automated FDE agent architecture.

Step 1 — Provision the environment

1. Deploy a private Vector Knowledge Graph (e.g., Greptile/Qdrant) connected to your enterprise GitHub/GitLab org.
2. Spin up a secure cluster (AWS ECS / GKE / Kubernetes) dedicated to hosting the orchestration engines (LangGraph/CrewAI).
3. Set up E2B or Docker sandboxes with restricted network access to prevent data exfiltration.

Step 2 — Configure agent guardrails

1. **Token boundaries.** Define strict spend per task (e.g., max $5.00 of LLM usage per ticket) to prevent infinite loops.
2. **Branch protection.** Agents may never push to main/production — they must submit a Pull Request.
3. **Static analysis.** Run SonarQube/Snyk automatically on every agent commit.

Step 3 — Monitoring & self-healing run loop

1. Watch telemetry for execution loops (e.g., an agent failing the same test more than 5 times).
2. If an agent loops, alert and drop the task into a human developer's triage queue.
3. For live crashes, ingest the log trace into a Self-Healing Agent webhook to auto-cut a bug-fix branch.

Multi-cloud deployment blueprints

Option A — AWS Native (Bedrock & ECS)

- **Context & ingestion:** sync repos to an Amazon Bedrock Knowledge Base backed by OpenSearch Serverless.
- **Orchestration:** deploy the multi-agent architecture to AWS Fargate or coordinate via Amazon Bedrock Flows.
- **Execution sandbox:** route agent actions into AWS Lambda ephemeral storage (/tmp up to 10GB) or isolated micro-VMs.

Option B — GCP Native (Vertex AI & Cloud Run)

- **Context & ingestion:** index repo files and docs into Vertex AI Vector Search.
- **Orchestration:** manage agent state with Vertex AI Agent Builder + LangGraph runtimes on GKE.
- **Execution sandbox:** run untrusted agent code inside Cloud Run v2 with sandboxed gVisor runtimes.

Option C — Azure Native (Azure AI & Container Apps)

- **Context & ingestion:** connect codebases and wikis to an Azure AI Search vector index.
- **Orchestration:** orchestrate multi-agent interactions with Azure AI Studio + Microsoft AutoGen.
- **Execution sandbox:** spin up Azure Container Apps Dynamic Sessions for fast, secure code-execution isolation.

Scalability best practices

Practice	What it does	Why it matters
State persistence	Save agent trace histories to a database (e.g., PostgreSQL).	Resume long-running tasks after a network error — no lost work.
Token budgeting	Hard limits on max LLM iterations per agent runtime.	Avoids runaway loops and unexpected billing spikes.
Semantic caching	Cache common agent queries and code snippets.	Cuts latency and API compute cost.
Context windows	Pass only relevant files or tree-map skeletons, not the whole repo.	Prevents context-window degradation and improves accuracy.

Best practices for maximum enterprise adoption

The "Opt-in" trust model. Market Embedded PODs first as a "super-powered intern" to standard application teams. Once they trust the agent's PR outputs, offer a toggle to upgrade the service to a Remote POD. Trust is earned in the Embedded tier and spent in the Remote tier — the same land-and-expand logic that governs forward-deployed work.

Unified telemetry. Pipe logs from all clouds (CloudWatch, Google Cloud Logging, Azure Monitor) into a central observability layer (Arize, Phoenix, or LangSmith) to audit agent behavior uniformly.

The "Kill Switch" guardrail. Every cloud environment must have an automated token/compute quota manager. If an agent loops past its hourly budget, automatically tear down the sandbox and alert a human SRE.

THE GOVERNING PATTERN

Across all seven stages, both topologies, and four clouds, one rule holds: the agent proposes, a deterministic gate disposes — a human gate in Embedded, an automated verification gate in Remote, a hard kill-switch in both. Cloud-agnostic core, cloud-native bindings, governed execution.